

Numerics & Codebase Tour

from Newton's method to the production solver

Onboarding doc 03 of 06

May 11, 2026

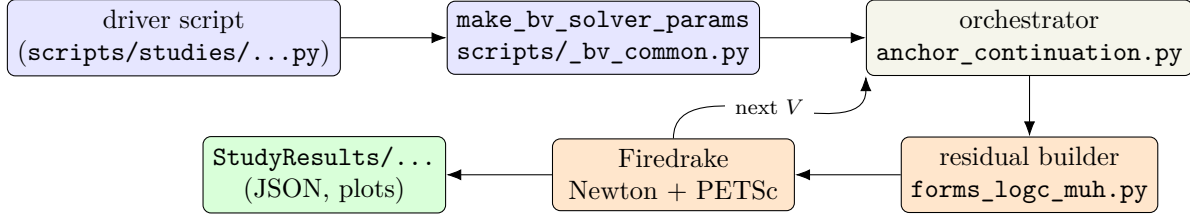
What this document is for

A guided tour of how the equations from doc 02 turn into running code. Covers the finite-element library (Firedrake), the nonlinear solver strategies (Newton + continuation + Picard), and the layout of the actual repository. By the end you should be able to find where any reaction term, IC seed, or solver flag lives.

Contents

1	The pipeline at a glance	1
2	Finite elements in 60 seconds	1
2.1	The mesh	2
3	Newton's method and where it goes wrong	2
3.1	Why cold-start often fails	2
3.2	Continuation: walk to the solution one step at a time	3
3.3	Picard for the outer loop	3
3.4	Adaptive ladders (step 9.5 onwards)	3
4	Repository layout	3
4.1	Inside <code>Forward/bv_solver/</code>	4
4.2	Inside <code>scripts/studies/</code>	4
4.3	Where the tests live	5
5	Anatomy of a study run	5
5.1	Outputs	5
5.2	The convergence flags	5
5.3	Cross-checking: validation pass	6
6	Writing a new diagnostic	6
7	The adjoint pipeline (paused, but useful background)	6
8	Hard rules summary	6

1 The pipeline at a glance



A driver script picks bulk concentrations, kinetic constants, and a voltage grid; it asks the factory for a packed `solver_params` object; it hands those to the orchestrator, which loops over voltages and continuation rungs; for each rung the orchestrator builds the FEM residual via the forms file and asks Firedrake to Newton-solve it. Converged solutions get post-processed into a JSON of observables and saved under `StudyResults/`.

2 Finite elements in 60 seconds

You saw the weak form briefly in doc 02 §9. The **finite-element method (FEM)** approximates each unknown function $u(x)$ as

$$u_h(x) = \sum_{j=1}^N U_j \varphi_j(x),$$

where $\{\varphi_j\}$ is a basis of **tent functions** (piecewise polynomial, supported on a few mesh elements each). The U_j are the unknown **degrees of freedom (DoFs)**. Plugging this into the weak form turns each PDE into an equation per basis function, giving a square nonlinear system $F(U) = 0$ for $U = (U_1, \dots, U_N)$.

Why FEM rather than finite differences? FEM handles non-rectangular domains, variable mesh resolution, and Robin boundary conditions natively. We do not need any of those features in the strict sense (we use a 2D rectangular mesh with graded refinement near the electrode), but FEM gives us: (i) trivial mesh refinement near the EDL singular layer; (ii) a well-developed adjoint capability via Firedrake’s `firedrake.adjoint` (Pyadjoint), which we will need for the inverse problem when it un-pauses; and (iii) symbolic assembly of the Jacobian for Newton, which would be brittle to write by hand.

Firedrake. The Python library that makes this all declarative. You write the weak form with operators like `inner(grad(u), grad(v))*dx`, declare a function space, and ask it to `solve(F == 0, u)`. Under the hood it generates C kernels for assembly (via TSFC), assembles the sparse Jacobian, and hands it to PETSc for the linear solve. You will not need to read the Firedrake internals; treat it as a tool.

2.1 The mesh

We use a 2D **graded rectangle** of width ~ 1 scaled unit and height $\hat{L} = 1$ (which corresponds to physical $L_{\text{eff}} \sim 16\text{--}100\ \mu\text{m}$). The graded grid puts most nodes near the electrode at $x = 0$ where the EDL lives:

$$y_j = \hat{L} \cdot \frac{e^{\beta j / N_y} - 1}{e^{\beta} - 1}, \quad \beta \approx 3,$$

giving ~ 80 nodes spaced over five decades of physical distance. The mesh constructor lives in `Forward/bv_solver/mesh.py` (`make_graded_rectangle_mesh`). **Keep $\hat{L}$ matched to whatever `l_eff_m` the solver was built with:** an $\hat{L}$ -mismatch silently puts the bulk Dirichlet BC at the wrong physical location.

3 Newton’s method and where it goes wrong

Newton’s method (you have seen it):

$$J(U^k) \Delta U = -F(U^k), \quad U^{k+1} = U^k + s_k \Delta U,$$

with line-search step $s_k \in (0, 1]$. Firedrake delegates the linear solve to PETSc; we use a sparse direct solver (MUMPS) at this problem size since the Jacobian fits in memory.

3.1 Why cold-start often fails

Two failure modes dominate:

1. **The Jacobian is poorly conditioned** when $|\eta|$ is large. The BV exponentials (doc 01) saturate to 10^{40} -ish numbers; pre-clipping helps but does not eliminate the problem. We use `exponent_clip = 100` (clip on $\eta_{\text{scaled}} = (V - E_{\text{eq}})/V_T$ before multiplication by αn_e); see `docs/solver/clipping_conventions.md`.
2. **The initial guess is too far from the solution.** For a target like $V_{\text{RHE}} = +1.0$ V at production k_0 , a cold-start from a uniform-bulk profile lands the Newton iteration on the wrong side of a manifold and diverges.

3.2 Continuation: walk to the solution one step at a time

The fix is **continuation**: replace the hard problem with a sequence of easier problems whose solutions are close together. We use three layered continuations:

Voltage continuation. Solve at a sequence of V_{RHE} values, using each converged solution as the warm-start initial guess for the next V_{RHE} . Implemented as the “warm-walk” phase of the orchestrator.

k_0 continuation. Within a single V_{RHE} , scale the kinetic prefactor down by many orders of magnitude ($10^{-12}, 10^{-9}, 10^{-6}, 10^{-3}, 10^0$ relative to target), solve, then scale back up. Tagged `initial_scales` in the code.

Phase 6β -only: λ_{hyd} and k_w^{eff} ladders. Smooth turn-on of the cation hydrolysis source (λ) and water self-ionization rate (k_w) via outer loops, similar to a homotopy from the Phase 5γ solution to the Phase 6β target.

The orchestrator that owns these loops is in `Forward/bv_solver/anchor_continuation.py`, specifically:

- `solve_anchor_with_continuation`: build a converged “anchor” solution at one well-chosen voltage, ramping k_0 and k_w^{eff} ladders to target.
- `extract_preconverged_anchor`: repack the anchor for warm-start use.

- `solve_grid_with_anchor`: warm-walk the converged anchor across a list of voltages.

This pair is the production entry point for the multi-ion + Stern + Phase $6\alpha/6\beta$ stack. The older `solve_grid_per_voltage_cold_with_warm_fallback` (“C+D” orchestrator) is used for the legacy single-counter-ion ClO_4^- stack but cold-fails on the multi-ion stack near $V_{\text{RHE}} \approx +0.55$ V (13/13 fail). **Do not pick C+D for deck-aligned work**; use the anchor-and-grid pair.

3.3 Picard for the outer loop

Newton converges quadratically near the solution but needs a smooth residual. The cation hydrolysis coverage equation (doc 02, $\dot{\Gamma}$ formula) has algebraic clamps and is not smooth enough for Newton to like it. We solve it with a separate outer **Picard iteration**: solve the inner PNP+BV Newton with Γ held fixed; update Γ from the new fields; repeat until Γ stops moving. The Picard helper lives in `Forward/bv_solver/cation_hydrolysis.py`.

3.4 Adaptive ladders (step 9.5 onwards)

Late Phase 6β added `AdaptiveLadder` in the `Forward/bv_solver/anchor_continuation.py`, which arithmetic-bisects the λ_{hyd} ramp when a Newton solve fails partway up. The `warm_start_floor` option in step 9.5 (May 2026) added the ability to bisect from the warm-start floor ($\lambda = 0$), rescuing 3/14 high- k_{hyd} configurations in the step 9 B.2 sweep.

4 Repository layout

A 30-second tour of the top of `PNPInverse/`:

Path	What lives there
<code>Forward/</code>	Forward solver code. The active surface.
<code>Forward/bv_solver/</code>	The PNP–BV Firedrake package. Most of your work will be here.
<code>scripts/studies/</code>	Driver scripts. One file per study.
<code>scripts/derive/</code>	Data-derivation scripts (Tafel slopes, etc.)
<code>scripts/verification/</code>	MMS and other verification tests.
<code>calibration/</code>	Top-level constants (<code>v10b.py</code>). Firedrake-free.
<code>Nondim/</code>	Physical constants & nondim transforms.
<code>tests/</code>	Pytest tests. <code>slow</code> -marked ones need Firedrake.
<code>docs/</code>	All written documentation.
<code>StudyResults/</code>	Output of every run we have kept. JSONs + plots.
<code>data/</code>	<i>Gitignored</i> . Experimental data (273 MB). Ask for the drop.
<code>Inverse/</code> , <code>FluxCurve/</code> , <code>Surrogate/</code>	<i>Paused</i> . Don’t touch.
<code>archive/</code>	Old code, reference only.
<code>CLAUDE.md</code>	The hard rules. Read once, refer often.
<code>README.md</code>	The project narrative.

4.1 Inside Forward/bv_solver/

File	Role
forms_logc.py	Residual builder, log-c on all species (legacy backend, kept for cross-check).
forms_logc_muh.py	Residual builder, μ_H on protons (production backend).
boltzmann.py	Bikerman analytic counter-ion expressions (multi-ion).
water_ionization.py	Phase 6 α $\text{H}_2\text{O} \rightleftharpoons \text{H}^+ + \text{OH}^-$ closure.
cation_hydrolysis.py	Phase 6 β Γ_{MOH} Picard + Singh pK_a shift.
anchor_continuation.py	Phase 5 γ orchestrator: anchor + grid + ladders.
grid_per_voltage.py	Legacy C+D orchestrator (ClO_4^- stack only).
picard_ic.py	Shared Picard outer loop for the <code>debye_boltzmann</code> IC.
dispatch.py	Routes <code>build_context</code> , <code>build_forms</code> , <code>set_initial_conditions</code> to the right backend.
config.py	Reads/validates the <code>bv_convergence</code> options.
diagnostics.py	Per-voltage diagnostics (BV-rate components, F_0 , θ , ...).
rrde_observables.py	Postprocesses to ring/disk currents, selectivity, n_e .
validation.py	W1–W5 sanity checks on a converged solution.
mesh.py	Graded rectangle mesh constructor.

4.2 Inside scripts/studies/

Each driver corresponds to one named study or one paper-figure target. The current ones to know:

Driver	What it does
phase6b_step6_plumbing_ablation.py	$\text{K}^+/\text{SO}_4^{2-}$ + cation hydrolysis at V_{kin} with Step 6 ablation flags.
phase6b_v10a_phase_A2_v_kin.py	Phase A.2 baseline reproduction at V_{kin} with v10a Langmuir cap.
phase6b_v10a_v_sweep_diagnostic.py	V-sweep diagnostic for V_{kin} selection.
l_eff_transport_sweep_csplus_so4.py	$\text{Cs}^+/\text{SO}_4^{2-}$ multi-ion + Phase 6 α validation.
mangan_full_grid_csplus_so4.py	$\text{Cs}^+/\text{SO}_4^{2-}$ deck page-15 V-band sweep.

4.3 Where the tests live

`tests/` contains pytest regression tests. Two markers: `not slow` (no Firedrake required) and `slow` (need the Firedrake venv active). Run them with

```
python -m pytest -m "not slow"
python -m pytest -m slow
```

Phase 6 β has its own test files (`test_phase6b_v9_*.py`, `test_phase6b_step10_phase_D_*.py`).

5 Anatomy of a study run

5.1 Outputs

A study run drops a folder under `StudyResults/<study_name>/` containing:

File	Contents
<code>iv_curve.json</code>	The current–voltage curve at each grid voltage; the headline observable.
<code>summary.json</code>	Per-voltage convergence flags, BV-rate components, surface concentrations, Γ .
<code>verdict.json</code>	PASS/FAIL of the study’s predefined acceptance gates.
<code>*.png</code>	Plots (cd, peroxide, surface pH, etc.).
<code>run.log</code>	Console output of the driver.

When you wonder “did anyone already run this?”, open the `StudyResults/` subdirectory and read the existing `summary.json`. **A full sweep can take minutes to hours** (production = ~ 30 min for $8 \text{ combos} \times 13 \text{ voltages}$); do not regenerate without checking.

5.2 The convergence flags

Each voltage point in `summary.json` has a `converged` flag and an `exit_status`. The most common values:

- `newton_converged`: rosy, residual L^2 below tol.
- `ladder_partial`: a continuation ramp made it part of the way; the recorded fields are at a partially-converged k_0 scale.
- `cold_fail`: cold-start Newton diverged; warm-fallback may or may not have rescued.
- `stagnant`: Newton stopped reducing residual.

5.3 Cross-checking: validation pass

`validation.py` runs a fast set of W1–W5 sanity checks on each converged field set. They check things like clip saturation (W1), counter-ion depletion (W5), BV-rate vs. Faraday’s law consistency. These are cheap; always look at them before believing a result.

6 Writing a new diagnostic

The most common first contribution from a new collaborator is adding a diagnostic plot or a new observable. The pattern:

1. Pick the quantity you want to compute (e.g. “volume-averaged pH in the diffusion layer”).
2. Write a function in `Forward/bv_solver/diagnostics.py` that takes `ctx` (the solver context, a dict of UFL functions) and returns a Python float.
3. Add a regression test under `tests/` that pins it on a reference field.
4. Wire the diagnostic into the driver script you care about.
5. Run the study; verify the new field shows up in `summary.json`.

The repo has dozens of examples of this pattern; `diagnostics.py` itself is a good model.

7 The adjoint pipeline (paused, but useful background)

`firedrake.adjoint` (Pyadjoint) provides reverse-mode automatic differentiation through Firedrake solves. When the inverse work resumes, this is what gives us $\partial L / \partial \theta$ where L is a misfit and $\theta = \{k_0, \alpha, \dots\}$. The hard rule for adjoints is:

Adjoint tape hygiene

Any unannotated cold-ramp or continuation work must be wrapped in `with adj.stop_annotating():` Otherwise the tape records intermediate ladder solves and the gradient is wrong / slow. This is in `CLAUDE.md` and in the inverse handoff `docs/inverse/CHATGPT_HANDOFF_10_LM_TIKHONOV_BASIN_GEOMETRY.md`.

You will not need to write adjoints in your first months. Just know that “annotate” and “stop_annotating” are jargon for “record this solve onto the tape so we can backprop through it later”.

8 Hard rules summary

These are the project’s `CLAUDE.md` hard rules, condensed:

1. **Use anchor-and-grid (or C+D for legacy ClO_4^-), not Strategy B.** Strategy B (`solve_grid_with_charge`) Phase-1 cold-fails on the logc + counter-ion stack.
2. **`exponent_clip = 100` is the only PC-trustworthy setting.** Lower clips create fictitious peroxide currents at high V_{RHE} .
3. **The IC and the residual must agree about steric saturation.** Bikerman IC + ideal residual cold-fails; same goes vice versa.
4. **Use the parallel $\text{R}_{2e} + \text{R}_{4e}$ topology** (Ruggiero 2022). Sequential R_0/R_1 is wrong and is kept only for backward-compat.
5. **Check data/EChem Reactor Modeling-Seitz-Mangan/ first.** Do not conjecture about the experimental physics. The hypothesis is cation hydrolysis at the polarized OHP, not sulfate, not carbonate, not water self-ionization.
6. **$C_S = 0.20 \text{ F/m}^2$ is the literature-anchored target** (Bohra–Koper–Choi consensus). Lower values were used historically without citation.